Architectural Strategy and Execution Plan for the HR Copilot System

Executive Strategic Mandate

The development of the HR Copilot system has reached a critical inflection point, currently operating under a severe, non-negotiable 48-hour deadline. The overarching objective is to deliver a highly capable "action layer" that consumes organizational requests from discrete human resources, ticketing, attendance, and recruiting platforms, automating the downstream operational processes by utilizing autonomous artificial intelligence[1]. At present, the engineering effort is dangerously bifurcated across two conflicting repositories. The original "ClosedAI" repository contains a robust but malfunctioning Model Context Protocol (MCP) integration layer built upon a modified OpenHands framework[1]. Conversely, the newer "HR-Copilot" repository was initiated in response to the MCP execution failures, bypassing the architectural complexity by focusing on the rapid deployment of interface features and hardcoded, pre-authored workflows[1].

The fundamental strategic dilemma centers on whether to invest the remaining 48 hours in stabilizing the dynamic MCP tool execution within the ClosedAI repository or to continue appending hardcoded mock features to the HR-Copilot repository. Based on an exhaustive analysis of the system architecture, the OpenHands framework underpinning the core agent, and the operational mandate of the enterprise HR platform, the analysis unequivocally indicates that pursuing the HR-Copilot repository's hardcoded feature expansion constitutes a severe strategic error. Adding rigid features merely expands an illusion of capability, resulting in a brittle system that remains entirely incapable of resolving dynamic business problems outside of manually authored scripts.

The immediate, high-priority mandate is to revert all engineering focus to the ClosedAI repository. The team must diagnose the precise points of failure within the OpenHands asynchronous event stream and the FastMCP schema translation layers, establishing a genuine, dynamic Reasoning and Acting (ReAct) loop. This comprehensive report provides an exhaustive diagnostic breakdown of the OpenHands MCP integration, an analysis of the architectural dissonance currently plaguing the frontend interface, and a highly granular execution roadmap designed to transform the HR Copilot from a static demonstration into a dynamically capable enterprise execution engine.

Business Context and the Illusion of Capability

The project mandate explicitly positions the HR Copilot not as a replacement for existing ticketing or recruiting platforms, but as the central operational execution layer operating upon a shared data backbone[1]. Platforms dedicated to ticketing, applicant tracking, and attendance generate data and surface requests autonomously[1]. The HR Copilot is designed to consume this shared data in real-time and orchestrate the complex, multi-step actions required to fulfill

those requests alongside human resources personnel[1].

The Operational Workflow Vision

An accurate understanding of the target operating model requires distinguishing between request generation and operational execution. The current enterprise ecosystem operates multiple discrete platforms that must be synthesized by the Copilot.

Platform Type	Functionality and Output	Copilot Action Responsibility
Recruiting Systems	Generates hiring-related actions, candidate data, and onboarding requests.	Collects personal info, orchestrates document generation, retrieves paperwork, and tracks signatures[1].
Ticketing Platforms	Creates helpdesk requests across HR, IT, accounts, and operations.	Drafts compliant replies, searches policy databases, coordinates multi-departmental actions[1].
Attendance/Leave Systems	Flags attendance issues, leave requests, and PTO accrual milestones.	Analyzes employee eligibility, adjusts Cosmos DB records, communicates with management[1].
HR Core Operations	Manages lifecycle events such as transfers, promotions, and compensation adjustments.	Computes salary deltas, verifies non-compete thresholds (e.g., RCW 49.62), and provisions updates[2].

To achieve this level of operational execution, the system relies on a dynamically merged MCP marketplace architecture. Tools are installed to the local environment at ~/.HRAgent/plugins/installed/ and subsequently merged into the agent's context window[2]. However, the current iteration of the system fails to achieve dynamic orchestration. While specific workflows—most notably the employee onboarding sequence—appear to function seamlessly in demonstrations, they do so entirely through a pre-authored sequence of events rather than through genuine agentic reasoning[1].

The Capability Gap

The core contradiction within the current system state lies between the architectural

philosophy outlined in the system design documents and the implementation reality present in the codebase. The overarching design philosophy demands an agent capable of receiving a high-level, ambiguous goal, formulating a plan, selecting appropriate tools from a dynamic registry, sequencing actions, and pausing for human-in-the-loop approvals before executing state changes[1]. Yet, apart from the core Chat interface, the broader system is fundamentally hardcoded.

The components visualized in the user interface—such as the Work Queue, Automations, and Intake pages—rely entirely on mock data and static pre-authored demonstrations[1]. The Work Queue items feature explicitly hardcoded steps and percentage completions that do not correspond to any live computational process[1]. The statistical figures on the Automations page are static integers, and the Intake dispositions operate on mock JSON arrays rather than live agent judgments[2].

Consequently, if a user attempts to execute a workflow outside of the rigidly defined onboarding script, the agent is entirely incapable of resolving the issue[1]. It cannot determine how to independently handle an employee transfer, evaluate benefits implications, or coordinate with the IT department because the agent's actual computational action space is confined to read-only lookups[1]. The system can successfully query employee records, check PTO balances, retrieve organizational charts, and execute semantic searches against Azure AI Search for policy documents, but it lacks the dynamic planning mechanisms to chain these read operations into a cohesive write-oriented workflow[1].

Adding more features to the HR-Copilot repository under the current paradigm simply involves writing additional manual scripts for specific edge cases. This approach lacks fundamental scalability. An enterprise organization cannot anticipate and manually code every variation of an employee relocation, leave of absence, or promotion cycle. To meet the business mandate, the system requires a genuine, retrievable skills layer representing procedural knowledge, a robust tool surface capable of extensive read and write operations, and persistent database execution capable of spanning multiple sessions[1].

OpenHands Architectural Evolution and the Event Stream

To definitively diagnose why the MCPs are experiencing intermittent failures within the ClosedAI repository, it is necessary to perform a rigorous examination of the underlying OpenHands framework. OpenHands operates as a leading open-source platform specifically designed for building autonomous AI agents that interact with external environments, codebases, file systems, and web interfaces[3]. The framework recently underwent a highly significant architectural redesign from its initial V0 state to a robust V1 iteration, driven primarily by the need to resolve the cascading complexities of monolithic designs and to natively support the Model Context Protocol[5].

The Shift from Monolithic to Composable Architecture

The original V0 architecture of OpenHands was driven by the necessity for rapid prototyping,

which inadvertently resulted in a tightly coupled, monolithic codebase[5]. Core agent logic, command-line interfaces, web servers for the graphical user interface, frontend react code, and diverse runtime providers coexisted within a single structure[5]. As the system accumulated complexity—particularly with the introduction of Large Language Model (LLM) structured tool use—this tight coupling generated immense architectural friction[5].

The V1 redesign introduced a strict separation of concerns through a composable, stateless architecture designed around four primary design principles[5]. All components, including the agents, tools, and language models, are immutable and strictly validated at the point of construction[5]. The agent core is entirely decoupled from the application layers, allowing downstream systems such as web user interfaces or API servers to utilize the core as a shared library rather than duplicating execution logic[5].

Architectural Component	Functional Description within V1 Framework
openhands.sdk	Contains the core abstractions including the Agent, Conversation, LLM, Tool, and the critical MCP event system[5].
openhands.tools	Houses the concrete tool implementations based strictly on the abstractions defined within the SDK[5].
openhands.workspace	Defines the execution environments, enabling the agent to run locally or remotely within secure, sandboxed containerized environments[5].
openhands.agent_server	Operates as the web server exposing REST and WebSocket APIs for remote execution and user interface communication[5].

The Immutable Event-Sourced State Model

The single most critical architectural pattern within the OpenHands ecosystem is its event-sourced state model. Because the agents themselves are completely stateless configuration objects rather than living entities, all mutable context and historical data must reside within a single, highly structured ConversationState object[3]. The autonomous agent operates essentially as a pure mathematical function, continuously mapping the historical event stream to deduce the next subsequent logical event[3].

This architecture manifests continuously as a deeply integrated Action-Observation loop. Every action the agent initiates, and every response it receives from a localized environment or

external tool, flows through this centralized chronological event stream[7].
The sequence operates under a strict computational contract:

1. **Action Proposal and Validation:** The Language Model generates a structured JSON request proposing to invoke a specific tool. This request is instantly intercepted and validated against a strictly defined Pydantic schema[6]. If the schema validates, the request is parsed into an immutable ActionEvent[6].
2. **Execution Routing:** The ActionEvent is then passed to a ToolExecutor. Because Python executors are non-serializable, OpenHands uses a registry-based resolution mechanism where tools are represented as lightweight specification objects containing a registered name and JSON-serializable parameters[6]. The ToolExecutor reconstructs the full definition at runtime and performs the underlying execution, whether interacting with a local filesystem or dispatching requests to an external MCP server[5].
3. **Observation Processing:** The result of the execution is captured by the system, structured according to a defined return schema, and converted into an ObservationEvent. This event is subsequently appended to the event stream, where it informs the language model's next logical deduction[6].

The Impact of Structural Modifications

The system modifications implemented in the ClosedAI repository provide the most vital forensic clue regarding the current MCP execution failures. The development team deliberately stripped the native code-generation functionalities out of the OpenHands base, physically moved the file structures, and attempted to manually re-hook the inputs to adapt the general-purpose framework specifically for HR operations[1].
This aggressive surgical alteration of the OpenHands framework almost certainly disrupted the fragile timing and routing of the Action-Observation loop. Because the event stream operates asynchronously, manually modifying the file structure or the underlying mechanisms that route these events can easily sever the vital connections between an emitted ActionEvent and its corresponding ObservationEvent[1]. The OpenHands core relies heavily on the strict separation of concerns, decoupling the agent core from the application interfaces via REST and WebSocket services[5]. If the re-hooking of inputs failed to properly configure the event subscriptions, the asynchronous callbacks, or the threading locks designed for process safety, the system will exhibit the precise symptoms currently observed: intermittent and highly unpredictable tool execution failures.

Model Context Protocol Integration Dynamics

The Model Context Protocol (MCP) operates as an open, standardized mechanism utilized by OpenHands to communicate seamlessly with external tool servers. This protocol allows developers to dynamically extend the agent's capabilities—enabling specialized data processing, external API access, and custom enterprise tools—without hardcoding tool logic directly into the agent's core repository[11]. Understanding the intricate mechanics of this integration is absolutely essential for diagnosing the execution flaws plaguing the ClosedAI repository.

Transport Protocols and Tool Registration

When an OpenHands instance initializes, it systematically reads the MCP configuration, which can be defined either programmatically via the SDK, through the mcp.json file, or via the config.toml structure[11]. The system then establishes persistent connections to the specified servers using one of three supported transport protocols:

1. **Server-Sent Events (SSE):** Utilized primarily for remote connections, allowing the server to push real-time updates to the OpenHands client over HTTP[11].
2. **Streamable HTTP (SHTTP):** A bidirectional streaming HTTP connection suitable for high-throughput tool interactions[11].
3. **Standard Input/Output (stdio):** Employed for localized tools, executing subprocesses directly on the host machine[11].

Following the establishment of transport layer connectivity, OpenHands dynamically interrogates the servers to discover available capabilities and registers the tools provided by these servers with the active agent instance[11].

FastMCP and Pydantic Schema Translation

The critical translation layer that enables a language model to successfully operate an external MCP tool involves FastMCP and Pydantic schema validation. The JSON Schemas exposed natively by the external MCP servers are automatically captured and translated by the OpenHands SDK into compatible Action models, which function as the required input schemas for the agent[6].

This auto-generation of LLM-compatible tool descriptions ensures absolute type safety. The arguments generated by the language model are validated against the Pydantic model prior to execution[5]. If the parameters contain missing fields or incorrect data types, the execution is blocked before it reaches the external server, preventing malformed requests from causing cascading system errors[5]. Upon successful validation and subsequent execution, the external MCP server returns a response. OpenHands captures this raw response and structures it as an ObservationEvent, seamlessly injecting it back into the persistent conversation history[3].

OAuth Authentication Constraints

The system configuration indicates the presence of Microsoft 365 (Graph API), Gmail, and LinkedIn integrations, alongside planned implementations for Slack, Jira, and GitHub[2]. Many of these enterprise integrations, particularly those operating via the Model Context Protocol, require OAuth authentication mechanisms rather than static API keys[11].

The OpenHands SDK, leveraging the FastMCP library, natively supports OAuth-based MCP servers[11]. However, this introduces a significant constraint: OAuth MCP servers require user interaction for the initial browser-based authentication flow[13]. The SDK initiates an OAuth flow, opening a browser window for the user to authorize access, after which the tokens are securely stored locally in ~/.fastmcp/oauth-mcp-client-cache/ and automatically refreshed[13]. This inherently means these tools are unsuitable for fully automated, headless background workflows unless the provider explicitly offers an API key authentication alternative[13]. The

failure of the system to correctly handle these browser-based interaction prompts during background agent runs is a highly probable cause for the continuous deferral and failure of the communication-layer MCPs[2].

Diagnostic Analysis of Current Implementation Failures

The status report meticulously details that within the ClosedAI repository, some MCP integrations are functioning perfectly while others fail sporadically. The Cosmos DB and Azure AI Search tools, which use static account-key authentication, are successfully queried by the agent[2]. The Cosmos DB integration is seeded with 517 employees across 19 containers, and the agent successfully utilizes the count_documents and query_cosmos tools[2]. Similarly, the Azure AI Search instance allows the agent to actively search and retrieve HR company policies using hybrid-search and semantic-search[2].

However, the Document Editor MCP, backed by office-oxide-mcp, exhibits mixed and unpredictable results. Tools such as office_fill_pdf_form successfully fill AcroForms, and office_read extracts data reliably. Conversely, office_analyze_pdf_layout garbles text extraction on specific PDFs like employee non-disclosure agreements, and office_template_detect entirely fails to support table-based placeholder formats in standard document files[2].

Furthermore, the system occasionally suffers from intermittent tool unavailability, where the agent hallucinated that the document-editor exposes no callable tools, despite identical tests passing minutes prior[2].

A partial-failure pattern where some tools succeed while others fail almost never indicates a fundamental limitation or flaw in the chosen framework[1]. Rather, it invariably points to a specific mismatch or bottleneck in the data transformation pipeline. To systematically resolve this within the tight timeframe, the diagnostic process must evaluate the five distinct stages of MCP tool execution.

The Five-Stage Execution Diagnostic

Execution Stage	Diagnostic Focus	Potential Failure Mechanism in ClosedAI Repository
1. Transport & Connection	Verify the vitality of the MCP server independently of the OpenHands orchestrator.	The SSE connection may be timing out for tools that require extensive processing time (e.g., PDF layout analysis). Short timeouts (1-30s) will cause complex calculations to

		drop before the Observation is safely returned to the stream[1].
2. Tool Discovery & Schema Registration	Ensure the target tool's schema is accurately registered with the LLM.	The aggressive stripping and file restructuring of the OpenHands base likely disrupted discovery paths. Complex nested parameters in tools like template_detect might fail Pydantic schema translation, causing the LLM to receive malformed tool descriptions[1].
3. Invocation & Argument Passing	Validate that LLM-generated arguments match the MCP server's stringent expectations.	Silent failures often manifest here. The LLM attempts to call a tool, but due to subtle schema translation errors, the arguments are slightly malformed. The MCP server rejects the call, resulting in an untracked no-op[1].
4. Event Stream Wiring	Trace the async conversion of the tool response into an ObservationEvent.	This is the highest-probability failure point. The OpenHands event stream relies on incredibly precise asynchronous timing. The restructuring likely broke the logic that injects the Observation back into the stream, causing fast tools (Cosmos DB) to succeed and slower tools (Document Editor) to orphan their ActionEvents[1].
5. Result Parsing &	Confirm the React Canvas	The tool executes perfectly

Interface Rendering	module can safely render the specific data shape returned.	at the server level, but the Next.js frontend lacks a corresponding generic component to render the specific output schema, causing a silent failure at the presentation layer[1].

Frontend Instability and Context Window Collapse

Beyond backend execution, the UI and context window mechanisms exhibit severe instability. The Next.js frontend is suffering from a Maximum update depth exceeded error, resulting in a React infinite re-render loop when switching into a conversation or when the Side Canvas dynamically populates[2]. This is a classic state synchronization issue, likely caused by a useEffect hook that continuously triggers re-renders when the Side Canvas attempts to parse incoming Server-Sent Events (SSE) payload chunks[2]. Furthermore, clicking "New Chat" fails to clear the Side Canvas or Activity Panel, leaving stale execution summaries and cross-contaminating user sessions[2].

Simultaneously, the backend suffers from context window collapse. The intermittent tool unavailability—where the agent suddenly believes no tools exist—is directly tied to the OpenHands Condenser system. To ensure the ever-growing conversation history fits inside the LLM's finite context window, the Condenser system algorithmically drops events and replaces them with highly compressed summaries[6]. On longer, multi-step tool chains requiring five or more tool invocations, the Condenser operates too aggressively[2]. It inadvertently summarizes away the critical system prompt injections that define the available MCP tools and the presence of workspace files (e.g., i9_form.pdf), effectively blinding the agent to its own capabilities mid-execution[2].

Observability and Telemetry: The MLFlow Imperative

Resolving complex, asynchronous event stream disconnects within a 48-hour window requires total systemic visibility. Because OpenHands agents run autonomously within sandboxed environments, human operators cannot watch every step in real time[4]. Relying on standard terminal output to trace lost ObservationEvents is highly inefficient and prone to error.

The team must immediately activate the native OpenTelemetry traces emitted by OpenHands and pipe them directly into an MLflow server[4]. MLflow Tracing provides absolute observability and governance over the agent's execution layer with minimal initial setup[4]. By starting a local MLflow server (uvx mlflow server), the engineering team gains access to a structured, highly searchable record of every action taken[4].

The MLflow integration will instantly expose:

- The exact system prompts and tool schemas injected into the agent's context window, allowing the team to verify if the Condenser is destroying tool definitions[4].
- The precise tools actually invoked by the agent, alongside their raw inputs and outputs,

eliminating the ambiguity of silent failures at the invocation stage[4].

- The exact latency of each individual step, immediately highlighting if a tool like the Document Editor is exceeding the configured timeout threshold[4].
- A granular token usage breakdown and corresponding execution costs via the AI Gateway[4].

Furthermore, MLflow Evaluation provides a comprehensive toolkit for assessing the quality of the agent's output systematically[4]. It provides over 60 built-in scorers and LLM judges that can evaluate output relevance, correctness, and tool call efficiency directly from the MLflow UI[4]. This allows the team to definitively prove the agent's efficacy during the final stakeholder presentation, relying on empirical evaluation metrics rather than staged demonstrations.

The 48-Hour Technical Execution Plan

To transform the ClosedAI repository from a flaky, unreliable prototype into a dynamically capable execution layer within the 48-hour deadline, the team must execute a highly disciplined, surgical engineering plan. This plan strictly abandons all superficial UI feature development in the HR-Copilot repository to focus entirely on architectural plumbing, event stream integrity, and generic rendering.

Hours 0-12: Diagnostic Plumbing and Observability

The entirety of the first phase must be dedicated to resolving the partial-failure state of the MCP integrations by establishing total visibility. The goal is not to add new capabilities, but to ensure the existing foundational tools (Cosmos DB, Azure AI Search, Document Editor, Gmail, HR Core) function with 100% reliability[2].

1. **Deploy MLflow Tracing:** Instantly spin up the MLflow server and configure the OpenHands environment variables to route all OpenTelemetry traces to the server[4]. Run the failing Document Editor tools and isolate the exact millisecond the trace diverges from a successful Cosmos DB query.
2. **Isolate the Event Stream Disconnect:** Utilize the get_unmatched_actions() method provided by the OpenHands Conversation API[9]. This critical function identifies ActionEvents in the event history that do not possess corresponding ObservationEvents or UserRejectObservations[9]. Identify the actions that are pending execution forever, proving that the asynchronous callback was severed during the prior architectural restructuring.
3. **Direct Transport Verification:** Bypass the LLM orchestrator entirely. Utilize direct HTTP scripts or standard input configurations to manually send a request to the Document Editor MCP server[1]. If the server responds correctly to a manual request but fails in the agent loop, the transport layer is secure, isolating the fault strictly to the OpenHands wiring layer.

Hours 12-24: Schema Stabilization and Context Window Tuning

Once the point of failure is isolated, the team must repair the translation layer and prevent the context window from collapsing during extended runs.

1. **Audit Pydantic Models:** Inspect the dynamically generated Pydantic models within the MLflow traces. Compare the JSON Schema exposed by the Document Editor server against the Action schema registered inside the OpenHands LLM context. Repair any dropped fields or overly strict typing constraints that the LLM is violating during argument generation[1].
2. **Configure the Condenser:** Review and significantly alter the Condenser configuration. Ensure that the system prompts containing the dynamic tool injections and the awareness of local workspace files are permanently pinned in the context window[2]. They must be explicitly excluded from the condensation logic. The agent must never lose sight of its foundational capabilities.
3. **Implement the filter_tools_regex:** If specific tools within the Document Editor MCP (such as office_template_detect) remain fundamentally broken and consistently poison the LLM's planning phase with execution errors, use the filter_tools_regex parameter in the OpenHands configuration to explicitly block them from the agent's view[2]. Shrinking the available toolset to only highly reliable, proven functions will dramatically improve the agent's focus and end-to-end success rate.

Hours 24-36: The Dynamic ReAct Loop for Employee Transfers

With the backend stabilized, the team must prove the system's dynamic capability by abandoning pre-authored scripts and forcing the agent to dynamically resolve a complex HR scenario.
Select an ambiguous business process that is currently not scripted within the mock automation platform, such as an inter-departmental employee transfer[1].

1. Provide the agent with a high-level prompt: "Handle the transfer of Employee X from the Engineering division to the Sales division."
2. The agent, utilizing the stabilized MCPs, must independently initiate the ReAct loop without a predefined workflow[1].
   - It must independently decide to call employee_lookup to retrieve current compensation and reporting structure[1].
   - It must query org_chart to map the new managerial hierarchy[1].
   - It must execute a policy_search against Azure AI Search to evaluate specific implications, such as the Washington State RCW 49.62 non-compete thresholds, calculating if the new salary delta crosses the mandated $126,858.83 W-2 threshold[2].
   - Finally, it must emit an action approval request to the user interface, pausing execution until a human operator authorizes the transfer communication[1].
3. Do not write a script for this process. The LLM must deduce these steps based solely on its system prompt, the procedural skills injected into its context, and the schemas of the available tools[1].

Hours 36-48: Generic UI Synthesis and State Management

The final phase addresses the severe frontend instability. The current UI architecture is dangerously fractured. The Work Queue and Automations pages use bespoke, hardcoded

Zustand state stores that cannot adapt to dynamic agent behavior[1].

1. **Unify the Interface via Event Stream Consumption:** The frontend must be unified strictly around the OpenHands event stream. Pipe the exact same Server-Sent Events (SSE) stream that powers the Chat interface into the Work Queue component[1].
2. **Implement Generic Step Rendering:** Treat the Work Queue not as a predefined list of steps, but as a live, dynamic log of ActionEvents and ObservationEvents. When the agent emits a tool call, the React UI generic renderer intercepts the event and displays it as a dynamic step block. When the Observation returns, the UI updates the block to complete[1]. This eliminates the need to author bespoke user interfaces for every possible automation.
3. **Resolve React Infinite Re-renders:** Refactor the useEffect hooks in the Next.js frontend to ensure they are not continuously triggering deep re-renders upon every partial chunk received from the SSE stream[2]. Implement proper state cleanup functions so that clicking "New Chat" definitively wipes the localized Side Canvas and Activity Panel state, preventing cross-contamination between isolated conversation sessions[2].

Stakeholder Presentation Strategy

Given the severe temporal constraints, the final system presented at the deadline will not contain dozens of pre-configured workflows. It will feature a limited number of fully functional, deeply dynamic execution loops. Therefore, the presentation narrative must be carefully managed to project absolute engineering rigor over superficial interface features.

The presenter must explicitly delineate between the "Live Agent Engine" and the "Target Operating Picture"[1].

During the demonstration of the Live Agent Engine, the team must showcase the Chat interface executing the dynamic employee transfer loop constructed during the 48-hour sprint. The presenter must openly and explicitly state that this is not a pre-authored script. They must emphasize that the agent is actively deciding to query Cosmos DB, cross-referencing the results with Azure AI Search, calculating legal thresholds based on retrieved policies, and formulating a resolution plan entirely autonomously[1]. This demonstrates true computational intelligence and validates the immense power of the MCP architecture.

When transitioning to the Intake, Work Queue, and Automations pages, the presenter must explicitly frame these views as the Target Operating Picture. Explain clearly that while the UI elements currently hold placeholder visualization data, they represent the precise dashboard format that will exist once the dynamic event stream is fully piped into all interface components for operations at an organizational scale[1].

By adopting this highly transparent, structurally honest framing, the team avoids the catastrophic risk of technical stakeholders unmasking a deceptive hardcoded script. Instead, the stakeholders are presented with a highly impressive, mathematically sound execution engine and a completely verifiable vision for its integration into the broader enterprise software ecosystem.

Works cited

1. message.txt
2. message (1).txt
3. Run OpenHands Self-Hosted on Ollama, Bedrock, or Any LLM, https://dev.to/lynkr/run-openhands-on-any-model-you-want-1mnd
4. Harness Your OpenHands Agent with AI Observability and ... - MLflow, https://mlflow.org/blog/mlflow-openhands/
5. The OpenHands Software Agent SDK: A Composable and ... - arXiv, https://arxiv.org/html/2511.03690v2
6. The OpenHands Software Agent SDK: A Composable and ... - arXiv, https://arxiv.org/html/2511.03690v1
7. AlexCuadron/SWE-Bench-Verified-O1-native-tool-calling-reasoning, https://huggingface.co/datasets/AlexCuadron/SWE-Bench-Verified-O1-native-tool-calling-reasoning-high-results
8. The Best Agent Harnesses for AI Builders in 2026 - Future AGI, https://futureagi.com/blog/best-agent-harness/
9. openhands.sdk.conversation, https://docs.openhands.dev/sdk/api-reference/openhands.sdk.conversation
10. The OpenHands Software Agent SDK - OpenReview, https://openreview.net/pdf?id=pzVmWs6yGq
11. Model Context Protocol (MCP) - OpenHands Docs, https://docs.openhands.dev/openhands/usage/settings/mcp-settings
12. Model Context Protocol (MCP) - OpenHands Docs, https://docs.openhands.dev/overview/model-context-protocol
13. Model Context Protocol - OpenHands Docs, https://docs.openhands.dev/sdk/guides/mcp
14. Tool System & MCP - OpenHands Docs, https://docs.openhands.dev/sdk/arch/tool-system
15. jufjuf/openhands-mcp: My custom MCP servers for ... - GitHub, https://github.com/jufjuf/openhands-mcp
16. Feature: OpenHands Coding Agent Skill — Model-Agnostic ... - GitHub, https://github.com/NousResearch/hermes-agent/issues/477
17. Backend Architecture - OpenHands Docs, https://docs.openhands.dev/openhands/usage/architecture/backend